WEEK 1

Large-Scale Data Computing

The Landscape & the AI-Augmented Engineer

Curricular unit **CDLE — Computação de Dados em Larga Escala** / Large-Scale Data Computing

Batch at Scale
Spark pipelines over huge datasets

Streaming
Kafka, Spark/Flink & exactly-once

Lakehouse
ACID tables that unify batch & streams

AI-Augmented
Assistants as research instruments

Teacher: António Teófilo • CDLE • MEIC/MEIM • ISEL

From Raw Data to Reliable Insight

Engineer + AI

Explore and design with the assistant · delegate the draft

you decide, verify and own

Raw Data

Terabytes of events, logs & records — arriving nonstop

Distributed Platform

Spark distributes compute, Kafka partitions streams, lakehouse tables keep it consistent

Reliable Insight

Correct, reproducible results delivered at scale

1. Volume, velocity, variety

2. Partitioned & replicated

3. Trustworthy at scale

This is the dual mandate of CDLE

Master **distributed data systems** — and the **AI-augmented engineering** practice that builds them.

Contents

Everything for week 1 — the course, the landscape of data at scale, and how we will work with AI

01 The Curricular Unit

Rationale & the dual mandate
Learning outcomes
Structure, assessment & bibliography
The AI Usage Protocol

02 The Landscape

The reference architecture: two paths, one platform
A brief history: MapReduce → lakehouse
Dimensions of scale: the three Vs · Parquet
Batch vs streaming
The modern platform ecosystem

03 The AI-Augmented Engineer

What changed in the practice
Strengths & failure modes
Delegate → verify → own
A structured working session

04 Lab 1 — TPC-DS on DuckDB

Generate → Parquet → explore → 99 queries
Experiments: repeated runs & thread scaling
Take-home: SF10 data and the report
Next week: distributed systems principles

What is CDLE?

Large-Scale Data Computing (Computação de Dados em Larga Escala) — where data engineering at scale meets AI-augmented practice

Why It Exists

- Every product now emits data at scale: terabytes a day is ordinary, not exceptional
- AI itself runs on this plumbing: training, feature and evaluation pipelines are large-scale data jobs
- Generative AI has changed how engineers build these systems
- Industry expects both competences together; CDLE is built on that dual mandate

What You Will Master

- Batch, streaming & lakehouse architectures at real scale
- Cloud-native platforms engineered for cost and energy, not just speed
- A disciplined AI-augmented engineering workflow
- AI assistants as instruments of scientific enquiry
- Correctness, reproducibility & intellectual ownership

The Semester's Pillars

Foundations | **Batch** | **Streaming** | **Lakehouse** | **Cloud & AI** | **Synthesis**

Learning Outcomes

Seven outcomes — four on distributed data systems, three on the AI-augmented engineer

Data Systems at Scale — LO1–LO4

- LO1 Foundations: partitioning, replication, consistency, CAP
- LO2 Batch pipelines with Spark: plans, shuffles, tuning
- LO3 Streaming with Kafka, Spark & Flink: windows, exactly-once semantics
- LO4 Platforms: lakes, lakehouses, storage & table formats

AI-Augmented Practice — LO5–LO7

- LO5 AI for research: literature, hypotheses, synthesis
- LO6 AI for code: generation, testing, review, validation
- LO7 Reliability, cost, energy & ethics of data + AI systems
- You must be able to explain and defend every artefact

Assessed Through

Project 40% | **Report 35%** | **Talk 5%** | **Defence 10%** | **Labs 10%**

Structure & Rhythm

13 Weeks · 6 Modules

Foundations | Batch | Streaming | Lakehouse | Cloud & AI | Synthesis

3.5 h / week

Weekly session
≈1 h 30 lecture + 2 h AI-augmented lab

45.5 h contact

13 sessions in 15 weeks
1 holiday + 1 buffer week

Teams of 2

Semester project
One theme, staged: proposal, system, report

✓ Lecture + lab every week

✓ AI assistant, declared per artefact

✓ AI usage log per deliverable

✓ Min. 8.00 per component, 9.50 overall

Assessment — Six Components

Component	Weight	What it assesses
Proposal & checkpoints (team)	15%	Theme by week 4; architecture review week 6; streaming review week 9
Final system (team)	25%	Working prototype: batch, streaming and lakehouse layers, reproducible
Research report (team)	35%	Question, method, evidence, limitations — every claim traceable
Project presentation (team)	5%	Week 13: problem, one result with evidence, what went wrong — time kept
Individual oral defence	10%	Week 13, per student; a report section you cannot explain scores zero
Laboratory portfolio (team)	10%	Weekly in-class labs and AI usage logs; complete portfolio gates submission
Minimum grade	9.50/20	Final weighted average. 8.00/20 in each fundamental component: all except the Laboratory portfolio (RPAC art. 23.º)

Teams of 2 · No final exam (RPAC art. 21.º, n.º 1 b) · Single-project UC: no grade improvement or repeat (art. 23.º, n.º 8) · Theme menu and proposal template in the course repo from week 2

Undisclosed AI use, or an artefact you cannot explain, is an academic-integrity violation. The AI usage log protects you.

Bibliography & Readings

Reference	Authors	What it gives you	Status · free alternative
DDIA	Kleppmann — O'Reilly	Foundations of distributed data	**Core** · Kleppmann's lecture series
Spark guide	Chambers & Zaharia	Batch engines, in depth	**Core** · Official Spark guides
Streaming Systems	Akidau, Chernyak & Lax	Time, windows & watermarks	**Core** · Akidau, Streaming 101 & 102
Fund. of Data Eng.	Reis & Housley	The end-to-end platform view	**Compl.** · —
Kafka guide	Narkhede et al.	Logs & event streaming	**Compl.** · Kafka docs & Kafka paper
Primary papers	MapReduce · Kafka · Delta	Original sources + the Iceberg spec	**Core** · Open access

Check the ISEL library catalogue and b-on for the books; the free alternatives cover the same material, and the papers and the Iceberg spec are open access.

…plus current vendor & model documentation on AI-assisted engineering — refreshed every edition, given the pace of the field.

The AI Usage Protocol

DISCLOSE

AI usage log

Prompts, model, purpose & degree of reuse — with every deliverable

VERIFY CODE

Tests first

AI code ships with tests you authored or audited yourself

VERIFY CLAIMS

Primary sources

Every AI-generated claim traced to a reference you checked

OWN IT

Explain & modify

You defend any artefact you submit, whatever its origin

REMEMBER

Assistants err

Hallucinated APIs and confident nonsense are routine

GUARDRAILS

Data privacy

No personal or confidential data pasted into prompts

FAIR PLAY

Declared tools

Any assistant, always declared — model & version in the log

PAYOFF

A graded skill

Disciplined AI use is assessed as a professional competence

Three pillars — transparency, verification, ownership. The protocol turns AI use from an integrity risk into a competence we grade.

Prompting & the AI Log

THE PROMPT

Context + constraints

how you brief the assistant decides what you get back

- **Be specific** name the engine, versions, data shapes and the exact result you expect
- **Show the data** a five-row sanitized or synthetic sample beats three paragraphs of prose
- **Set constraints** “PySpark 3.5 (course-pinned LTS), no UDFs, must scale to 2 TB” changes the answer entirely

THE AI USAGE LOG

Every deliverable

the record that makes AI use a competence, not a risk

- **What & why** prompts used, model and version, and the purpose of each interaction
- **Degree of reuse** generated, adapted, or merely reviewed — declared per artefact
- **Verification** how you checked it — tests you wrote, sources you traced, proofs you ran

Golden rule: ***if you cannot explain it, you cannot submit it.***

Undisclosed AI use — or an artefact you cannot defend and modify — is treated as an academic-integrity violation.

Scenario — One Day of Clicks

A simple business question that no single machine can answer in time

The Question

"Which products trended in each country yesterday" / "Is fraud happening right now?"

- Yesterday's clickstream: ≈2 TB — about 4 billion events
- One machine scans ~200 MB/s — a single pass takes ≈2.8 hours
- Fraud detection cannot wait for tomorrow's batch — it needs seconds
- And the answer must be correct, repeatable and auditable

The Napkin Math

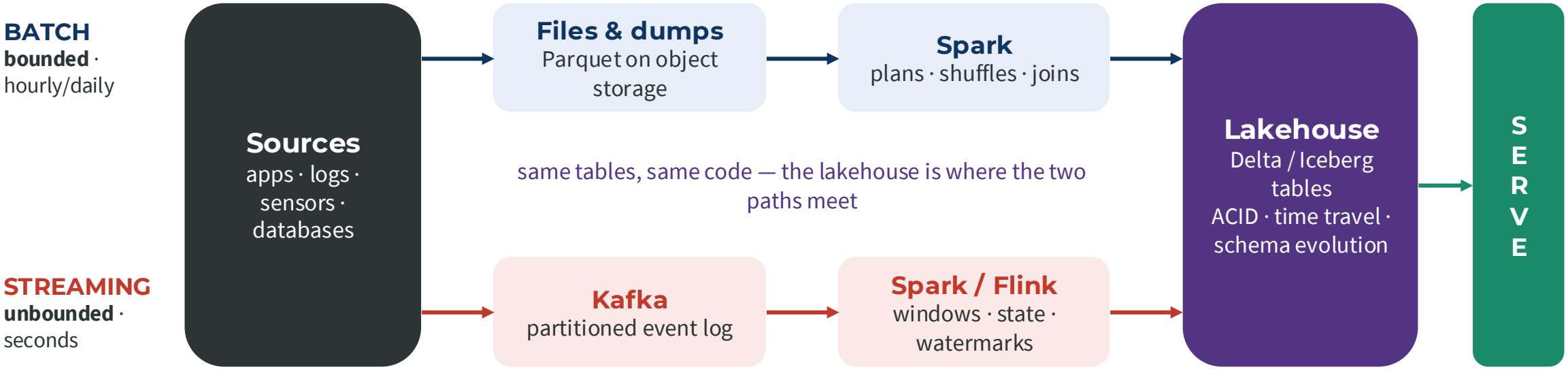
```
# one machine:
#   2 TB ÷ 200 MB/s ≈ 2.8 h per pass
#   one disk failure → start over
# 100 machines, in theory: ≈ 100 s
the catch — someone must:
partition data across the nodes,
  ship compute to the data,
  survive failures mid-job,
  and aggregate correctly = hard
→ that is what the engines do
```

The lesson

Scale is not a bigger laptop — it is a different discipline. **Distribution** buys speed but charges coordination, and **frameworks** exist to pay that cost for you.

The Reference Architecture at Scale

One platform, two paths — every system this semester is a piece of this picture



Partition the data
split storage and work across machines — the only way past one node's disk and CPU

Move compute to the data
ship the job to the partitions; shuffles are where time and money go

Tolerate failures
replicated logs, replayable input, transactional commits — correctness survives a crash

`Storage: week 3 · Batch: weeks 4–6 · Streaming: weeks 7–8 · Lakehouse: week 9 · Cloud & AI infra: weeks 10–11`

How We Got Here

Two decades of large-scale data computing — from MapReduce to the lakehouse

1

2003–04 · GFS & MapReduce

Google shows commodity clusters can store and process web-scale data. Hadoop (HDFS + MapReduce) brings the model to everyone (2006).

2

2009–14 · Spark

Born at Berkeley, Apache top-level in 2014. In-memory, general-purpose computation: 10–100× faster than MapReduce on iterative workloads.

3

2011–16 · Kafka & stream engines

The distributed log turns data into event streams (2011); Flink (2014) and Structured Streaming (2016) process them.

4

2018–21 · The lakehouse

Iceberg (2018) and Delta Lake (2019) bring ACID tables, time travel and schema evolution to open data lakes; the lakehouse is named in 2021.

Why this matters

- Every “new” platform recombines the same ideas: partition the data, move compute to it, tolerate failures.
- We will read the MapReduce, Kafka and Delta Lake papers and the Iceberg table spec during the semester.

Dimensions of Scale — The Three Vs

1 — Volume
terabytes → petabytes

2 — Velocity
millions of events per second

3 — Variety
tables, logs, media, graphs

Three pressures, one platform

```
// volume — what we keep
events ≈ 2 TB / day → 2 PB over 3 years
kept in parquet (columnar, compressed)
  storage is cheap — full rescans are not
  → partition pruning & file statistics

// velocity & variety — what arrives
peak ≈ 1.2 M events/s on campaign days (illustrative)
shapes JSON · logs · media · graphs — evolving
  late: events arrive out of order, by minutes
  → event time ≠ processing time
```

Why one V never comes alone

- ✓ More users bring more events and more formats
- ✓ Streams must land beside the batch data they extend
- ✓ Schemas evolve while the system keeps running
- ✓ Cost and energy grow with every V
- ✓ The architecture must absorb all three at once

"Big data" is any data that breaks the tools you have — the three Vs tell you which tool breaks first.

Parquet — Store by Column, Read Less

Row layout — the .dat files one strip per record

ss_item_sk	ss_qty	ss_price	ss_date_sk
1017	3	**12.40**	2451545
2201	1	**45.00**	2451545
1017	2	**12.40**	2451546
3390	5	**3.15**	2451547

*a query for **ss_price** still reads every byte of every strip*

Column layout — one Parquet file one strip per column

ss_item_sk	1017	2201	1017	3390
ss_qty	3	1	2	5
ss_price	**12.40**	**45.00**	**12.40**	**3.15**
ss_date_sk	2451545	2451545	2451546	2451547

*the same query reads only the **ss_price** strip (min 3.15 · max 45.00 lets DuckDB skip blocks)*

Why it matters

- ✓ **Read only the columns you use**
  TPC-DS queries touch a handful of store_sales' 23 columns; the rest is never read from disk.
- ✓ **Compress what looks alike**
  Same-typed values → dictionary & run-length encoding, then zstd. Lab: 1.2 GB of .dat becomes ≈0.3 GB.
- ✓ **Skip whole blocks**
  Each row group stores min/max per column, so a filter on ss_date_sk prunes blocks — and files — that cannot match.
- ✓ **Types travel with the data**
  Schema in the file. The .dat files have none: to_parquet.py must read tpcds.sql to know them.

Trade-off: built for scans, not for single-row updates — the lakehouse formats (week 9) add that on top of Parquet.

Open format, every engine reads it: DuckDB today, Spark from week 4; Delta and Iceberg tables are Parquet files plus metadata.

Key Concepts

```
# a batch pipeline over billions of rows
df = spark.read.parquet("s3a://events/2026/")
daily = (df.filter(col("status") == "OK")
    .groupBy("day", "country")
    .agg(count("*").alias("views")))
(daily.write.format("delta")
    .save("s3a://gold/daily"))

# the same API runs on one laptop
# or 1,000 nodes — the engine handles
# partitioning, shuffles & failures
```

The Vocabulary

- **Batch** → process a bounded dataset, end to end
- **Streaming** → process events as they arrive
- **Partitioning** → split data & work across machines
- **Shuffle** → move data between stages — costly
- **Lakehouse** → ACID tables on a data lake
- **Exactly-once** → each event affects the result once, even after failures

You will write code like this from week 4 — first we understand what the engine does underneath.

Batch vs Streaming

Two processing modes, one set of principles — this course covers both in depth

Batch — bounded data

```
# all data exists before the job
(spark.read.parquet(
  "s3a://events/2026/")
  .groupBy("day").count()

.write.save("s3a://gold/daily"))
```

Streaming — unbounded data

```
# events keep arriving, forever
(spark.readStream.format("kafka")
 .option("subscribe","ev").load()
 .withWatermark("ts", "10 min")
 .groupBy(window("ts","1 min"))
 .count())  # + sink & start
```

The practical difference

```
input    = bounded vs endless
latency  = mins-hours vs secs
hard bit = event time & state
engine   = can be the same
```

✓ One engine, both modes

✓ Windows & watermarks

✓ Exactly-once possible

*Both share the same foundations. **Batch** favours throughput and easy reprocessing; **streaming** favours latency and demands event-time care. The lakehouse (week 9) unifies them.*

Three Modes, One Comparison

	Batch	Streaming	Lakehouse
`Data scope`	Bounded: the dataset exists before the job	Unbounded: events keep arriving	One table layer serving both
`Latency`	Minutes to hours	Milliseconds to seconds	Both, on the same tables
`Hard part`	Shuffles, skew and cost at TB scale	Event time, state and late data	Consistency and compaction under mixed loads
`Typical job`	Nightly reporting, model training	Fraud detection, live monitoring	One source of truth for reports and streams
`Primary goal`	Deep historical analysis	Immediate operational action	Consolidate into a single data layer

Detail follows in weeks 4–6 (batch), 7–8 (streaming) and 9 (lakehouse); the ecosystem slide next maps the tools to each mode.

The Ecosystem at a Glance

The engines & platforms

Apache Spark
batch & streaming compute

Apache Kafka
the distributed event log

Delta / Iceberg
ACID tables on the lake

Apache Flink
low-latency stream engine

Airflow
pipeline orchestration

Where each one fits



Store
Object storage + Parquet holds raw and refined data cheaply



Move
Kafka delivers events to every consumer group



Process
Spark & Flink transform data in bulk or in real time



Serve & govern
Lakehouse tables add ACID, table history and time travel to queries

Today's lab uses none of these: DuckDB on one machine is the baseline. Spark from week 4, Kafka week 7, cloud week 10.

The AI-Augmented Engineer

✓ What the assistant gives you

- **Speed** — boilerplate, tests & refactors in seconds
- **Breadth** — wide coverage of APIs, papers & idioms, versions unchecked
- **Critique** — a sparring partner for designs & queries
- **Momentum** — idea to running prototype in one session
- **Clarity** — explanations & docs generated on demand

	Manual only you write every line	Blind AI use copy-paste, no checks	AI-augmented delegate · verify · own
SPEED	✗ Slow	✓ Fast	✓ Fast
TRUST	✓ You wrote it	✗ Unverified	✓ Tested
OWNED	✓ Fully	✗ Can't explain	✓ Fully

The assistant multiplies your productivity — the protocol keeps it from multiplying your mistakes.

The Working Agreement

How AI is used in every CDLE lab and deliverable — a contract modelled on production practice

Fair game to delegate

```
scaffolding — configs, compose files, glue
drafts — first versions of code & docs
refactors & tests you then audit
  /* critique of your designs */
triage of literature & related work
  /* summaries you spot-check */
→ always followed by your review
```

Always yours

- Requirements and the final design call
- Verification — the tests, benchmarks and source checks
- The AI usage log, and the honesty of what it declares
- The defence — explain and modify anything you submit

One rule of thumb for every artefact:

Delegate | **Verify** | **Own** | **Disclose**

One Loop for Every Artefact

Code, experiments, reports — the same four moves every week, logged every time

1. Specify
State the goal, constraints & tests first

2. Delegate
Brief the assistant with real context

3. Verify
Run the tests; trace claims to sources

4. Own
Log the usage; be ready to defend it

ai-log.md — one entry per artefact or session

```
- when: lab 1 · model: <assistant, version>
  purpose: draft of the top-N job
  reuse: adapted (≈60%)
  verified_by:
      "pytest 11/11 + explain() review"
  owner: I can modify & defend every line
// appended to each deliverable
```

Failure modes to avoid

- **Prompt-and-paste** — submitting output without tests is an integrity violation
- **Vibes-based trust** — “it looks right” is not verification
- **Silent AI use** — an empty log next to polished code convinces no one
- **Lost ownership** — if you cannot modify it live, you do not own it

PART 2 · HANDS-ON · TPC-DS ON ONE MACHINE

Lab

01

Lab 1 — TPC-DS on a Single Machine

1 · Generate
dsdgen · 1.2 GB .dat

2 · Convert
Parquet via DuckDB

3 · Explore
show_query.py

4 · Queries
dsqgen · 99 SQL files

5 · Measure
run_queries.py

Class plan — 100 minutes

0–10 min Intro: the TPC-DS data model and scale factors
10–20 min Part 1 — Build the tools
20–30 min Part 2 — Generate the 1 GB data
30–40 min Part 3 — Convert to Parquet
40–45 min Part 4 — Explore the data
45–60 min Part 5 — Generate and run the 99 queries
60–90 min Part 6 — Experiments
90–100 min Part 7 — Discussion

```
Before the lab — handout §0, required
✓ 4+ cores · 8 GB RAM · 25 GB free disk
✓ macOS / Linux / WSL 2 (files under ~)
✓ TPC-DS Tools v4.0.0 in ~/tpcds/DSGen-…-4.0.0
✓ build tools + venv with DuckDB
✓ starter pack in ~/tpcds/starter/
✓ machine table (§0.5) filled in

# in every new terminal:
export T=~/tpcds
export PY=$T/venv/bin/python
```

In class: SF1 (≈1 GB). Take-home: SF10 (≈10 GB), reused in Lab 2 on two machines. The handout is the reference; these slides list the actions.

Parts 1–2 — Build & Generate

Part 1 · Build the tools — 10 min · handout §1

```
cd $T/DSGen-software-code-4.0.0/tools
# macOS only: create macstub/values.h and malloc.h first (§1)
make -f Makefile.suite OS=LINUX \
  LINUX_CFLAGS="-O3 -Wno-… (see §1)"   # macOS: CC=clang -Imacstub
ls -l dsdgen dsqgen tpcds.idx      # ✓ the three files you need
```

Part 2 · Generate the SF1 data — 10 min · handout §2

```
mkdir -p $T/data/sf1; N=4
time ( for i in $(seq 1 $N); do
  ./dsdgen -SCALE 1 -DIR $T/data/sf1 -PARALLEL $N -CHILD $i \
           -TERMINATE N -QUIET Y &
done; wait )
du -sh $T/data/sf1                          # ✓ about 1.2 GB
cat $T/data/sf1/store_sales_*.dat | wc -l   # ✓ 2880404
```

Check & record

Part 1

- ✓ Many compiler warnings are normal; the three files are the check.
- ✓ The flags exist because the kit's C is old and macOS lacks values.h / malloc.h (Appendix B).

Part 2

- ✓ Run from tools/: dsdgen reads tpcds.idx from the current folder.
- ✓ Big tables come in 4 pieces (store_sales_1_4.dat …), plain text with | separators.
- ▸ **Record: real time, total size, number of files.**

Keep the short path with no spaces: dsdgen crashes past 200 characters of arguments and dsqgen truncates paths at about 80 (Appendix B).

Parts 3–4 — Convert & Explore

Part 3 · Convert to Parquet — 10 min · handout §3

```
$PY $T/starter/to_parquet.py \
    --kit $T/DSGen-software-code-4.0.0/tools \
    --src $T/data/sf1 --dst $T/parquet/sf1 --threads 4
du -sh $T/parquet/sf1        # ✓ every table prints OK (25 tables)
# types from tpcds.sql → DuckDB → zstd Parquet, folder per table
```

Part 4 · Explore the data — 5 min · handout §4

```
$PY $T/starter/show_query.py --parquet $T/parquet/sf1 \
    $T/starter/examples/02_small_table.sql
# run at least these four:
#  02 a whole dimension table (ship_mode, 20 rows)
#  05 a few rows of the store_sales fact table
#  06 numeric date key → real date through date_dim
#  07 fact ⋈ 2 dimensions: revenue per item category, 2000
--sql "select …"   --out file.csv   --max-rows 50   # other options
```

Check & record

Part 3

- ✓ All 25 tables OK; row counts match Appendix A.
- ▸ **Record: conversion time and Parquet size. Why is it so much smaller than the .dat files?**

Part 4

- ✓ Fact tables (store_sales…): millions of rows, keys to dimensions. Dimension tables (date_dim, item…): small, descriptive.
- ✓ Report item 3 comes from here: one example of each.
- ▸ **Optional: 09_top_states.sql — the 5 US states with most customers.**

Hint for the optional exercise: customer.c_current_addr_sk is a key into customer_address; the state is customer_address.ca_state.

Part 5 — Run the 99 Queries

Part 5 · Fix → generate → split → first run — 15 min · handout §5

```
# 5.1 fix the query templates (works on a copy; kit untouched)
bash $T/starter/fix_templates.sh $T/DSGen-software-code-4.0.0 \
     $T/query_templates
# 5.2 generate query_0.sql for SF1 (from tools/)
cd $T/DSGen-software-code-4.0.0/tools; mkdir -p $T/queries/sf1
./dsqgen -DIRECTORY $T/query_templates \
  -INPUT $T/query_templates/templates.lst -DIALECT netezza \
  -SCALE 1 -OUTPUT_DIR $T/queries/sf1 -QUIET Y
# 5.3 one file per query: query01.sql … query99.sql
$PY $T/starter/split_queries.py $T/queries/sf1/query_0.sql \
    $T/queries/sf1/split
# 5.4 first run
mkdir -p $T/results
$PY $T/starter/run_queries.py --parquet $T/parquet/sf1 \
    --queries $T/queries/sf1/split --out $T/results/sf1_first.csv \
    --threads 4                                    # ✓ OK: 99/99
```

Check & notice

- ✓ OK: 99/99 is the check. The results CSV has one line per query: rows, times, median, error.
- ✓ “Scale factor not valid for publication” is expected.
- ✓ 0 rows at SF1 is normal: random filters, and LIMIT 100 is a maximum.
- ✓ netezza dialect writes LIMIT n, which DuckDB reads; queries 14, 23, 24, 39 hold two statements.
- ▸ **Open fix_templates.sh: you explain two of its fixes in the report (item 4).**

AI-log entry #1: ask your assistant to explain two fixes in fix_templates.sh, check its answer against the templates and the DuckDB docs, log it.

Part 6 — Experiments (30 min)

A · Repeated runs — required, 10 min · handout §6A

```
$PY $T/starter/run_queries.py --parquet $T/parquet/sf1 \
    --queries $T/queries/sf1/split --threads 4 --runs 3 \
    --out $T/results/sf1_t4_runs3.csv  # compare run 1 vs 2 & 3
```

B · Thread scaling — required, 20 min · handout §6B

```
for t in 1 2 4; do   # add your core count if > 4
  $PY $T/starter/run_queries.py --parquet $T/parquet/sf1 \
      --queries $T/queries/sf1/split --threads $t --runs 3 \
      --out $T/results/sf1_t$t.csv
done
# T(t) = sum of medians printed by the script
# S(t) = T(1) / T(t)              speedup, whole set
# p    = (1 - 1/S) / (1 - 1/t)    Amdahl's parallel fraction
# same S(t) for q14, q23, q67 alone → median_s column
```

Questions to answer

A · Repeated runs

- Is run 1 slower? By how much?
- What is cached after run 1, and where (OS, DuckDB)?
- Why report the median, not one run?

B · Thread scaling

- Is the speedup close to linear? Why or why not?
- Queries take milliseconds at SF1: what share is not query work (parsing, planning, files, Python)?
- Do the heavy queries 14, 23, 67 scale better than the set? Why?

Fill in the table in §6B: T(t), S(t), p, q14, q23, q67.

Before measuring: plug in, close heavy apps, hands off. Optional C: .dat vs Parquet with --csv on queries 3,7,42,52,55,67. Optional D: --explain 67.

Part 7, Take-Home & Report

Part 7 · Discussion — 10 min

- Which step of the pipeline took longest, and what limited it: CPU, disk or memory?
- Same data and queries on two machines: what changes, and what new costs appear?
- Which of your results will change most in Lab 2?

Your answer to the last one becomes report item 9.

Take-home · SF10, before Lab 2

1. **Generate SF10:** dsdgen -SCALE 10 -DIR $T/data/sf10 (6–12 min, 11 GB)
2. **Convert:** --src data/sf10 --dst parquet/sf10 (≈1 min, 2.7 GB); counts vs Appendix A
3. **Queries:** dsqgen -SCALE 10 + split into sf10 folders; reuse $T/query_templates
4. **Repeat** experiments A and B at SF10
5. **Keep** parquet/sf10 and queries/sf10 for Lab 2; data/sf10 may go

8 GB machines: up to 8.3 GB used. Use -- runs 1 three times and say so in the report.

Report · max 4 pages, PDF

1. **Machine** — table §0.5
2. **Pipeline** — times and sizes, SF1 and SF10; all counts matched
3. **Data model** — fact vs dimension, one example each
4. **Query fixes — two, from fix_templates.sh, plus AI-log entry #1**
5. **Experiment A** — SF1 and SF10, explained
6. **Experiment B** — speedup plot + Amdahl p per scale
7. **1 GB → 10 GB** — why not exactly 10×
8. **Optional parts** — exercise, C and/or D
9. **Prediction for Lab 2** — one paragraph

TPC rules: download the kit yourself; never publish the kit, the data or the queries. Your numbers are derived from TPC-DS, not official results.

Key Takeaways

"Master the systems that process data at scale — and the disciplined AI workflow that builds them."

✓ What we saw today

- When one machine can't meet the deadline, distribute
- MapReduce, Spark, Kafka, lakehouse — not a succession
- Volume, velocity & variety rarely arrive alone
- Batch and streaming share one set of principles
- AI assistants are powerful — and only as good as your audit

✓ Your action plan

- Generate SF10 at home: data, Parquet and queries
- Keep parquet/sf10 and queries/sf10 — Lab 2 starts from them
- Submit the Lab 1 report (4 pages) with AI-log entry #1
- Skim DDIA chapter 1 — foundations begin next week
- Next week: partitioning, replication, CAP & PACELC

Thank you — welcome to CDLE!